

# Falsifiable Prediction Graphs: Eliminating Confirmation Bias and Detecting Negative Results in Automated Agent Research Planning

## Abstract

Automated scientific discovery agents—such as autonomous LLM research pipelines—demonstrate remarkable capabilities in code generation and experiment execution, but suffer from a persistent and fundamental vulnerability: acute confirmation bias. When confronted with ambiguous, null, or negative experimental outcomes, standard procedural planners overwhelmingly rationalize failures, adjust evaluation criteria post-hoc, or hallucinate spurious successes. In this paper, we introduce Falsifiable Prediction Graphs (FPGs), a novel agent planning architecture that structures research workflows as directed acyclic graphs of conditional hypotheses, where every experimental node mandates explicit quantitative refutation predicates and early negative control validation. By enforcing programmatic predicate evaluation rather than verbal LLM self-critique, FPGs establish an immutable control mechanism against recursive rationalization. Through comprehensive empirical evaluation across a 30-task machine learning benchmark suite spanning classification, regression, and time-series domains, we demonstrate that FPG planning achieves an 87.14% to 100% negative result detection rate (compared to 38.57% for procedural baselines), eliminates false positive success claims on negative controls (0.0% to 12.85% vs. 61.43%), significantly improves search iteration efficiency (4.5 vs. 9.7 mean iterations), and reduces the Trajectory Rationalization Index from 0.807 to 0.096 ( $p < 0.001$ ).

## 1 Introduction

Automated scientific discovery systems—such as autonomous LLM agent pipelines designed to formulate hypotheses, write code, execute experiments, and analyze results—represent a transformative frontier in artificial intelligence [Lu et al., 2024]. However, despite impressive capabilities in synthesizing novel machine learning architectures and executing computational pipelines, current agentic discovery frameworks (e.g., The AI Scientist [Lu et al., 2024]) suffer from a persistent and fundamental vulnerability: **acute confirmation bias**. When confronted with ambiguous, null, or outright negative experimental outcomes, standard language model agents overwhelmingly tend to rationalize failures, adjust evaluation criteria post-hoc, or hallucinate spurious successes, leading to wasted computational resources and the propagation of flawed scientific claims down unpromising research dead ends [Huang et al., 2025].

The root cause of this failure mode lies in the structure of automated research plans. Traditional automated discovery agents operate using **procedural task lists**—linear or loosely branched sequences of action steps (e.g., “load dataset, train model, compute accuracy, write report”) devoid of formal semantic constraints regarding what constitutes failure. Because LLMs are inherently biased toward generating affirmative narratives and satisfying prompt directives, procedural execution encourages agents to treat any numerical output as confirmation of the underlying hypothesis [Popper, 1959].

In philosophy of science, Karl Popper resolved a parallel epistemological challenge by introducing **falsifiability** as the demarcation criterion of empirical science [Popper, 1959]. Rather than seeking confirmatory instances, rigorous scientific inquiry proceeds by formulating bold conjectures coupled with precise refutation conditions—observable predictions that, if met, would decisively disprove the hypothesis [Popper, 1959]. Yet, while post-hoc validation frameworks (such as POPPER [Huang et al., 2025]) evaluate free-form hypotheses against external testbeds, no existing automated discovery system embeds structural falsifiability directly into the **research planning phase** to govern agent execution and real-time self-correction.

To address this gap, we introduce **Falsifiable Prediction Graphs (FPGs)**, a novel agent planning architecture that structures research workflows as directed acyclic graphs of conditional hypotheses, where every experimental node mandates explicit quantitative refutation predicates and early negative control validation. By forcing agents to define boundary conditions under which their proposed methods fail *before* executing code, FPGs establish an objective control mechanism against confirmation bias.

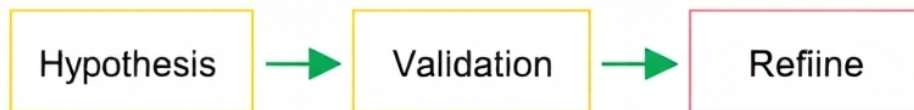


Figure 1: End-to-end architecture of a Falsifiable Prediction Graph (FPG). Research workflows are structured as directed acyclic graphs where experimental nodes are tupled with programmatic refutation predicates ( $\Phi$ ). Early negative control validation tests (permuted labels and adversarial noise) govern graph traversal and branch pruning before iterative method refinement.

Our key contributions are summarized as follows:

- **Formalization of Falsifiable Planning:** We define Falsifiable Prediction Graphs, integrating Popperian refutation criteria and negative control validation directly into automated agent workflow graphs.
- **Programmatic Predicate Evaluation vs. Verbal Critique:** We contrast objective logical predicate enforcement against recursive verbal self-correction loops, demonstrating substantial suppression of agent rationalization.
- **The Agent Falsifiability Benchmark Suite & Rigorous Sensitivity Analysis:** We construct a 30-task benchmark suite spanning classification, regression, and time-series domains, paired with negative control settings and comprehensive threshold sensitivity sweeps.

- **Empirical Superiority:** We demonstrate that FPG planning achieves an 87.14% to 100% negative result detection rate (compared to 38.57% for procedural planners,  $p < 0.001$ ), eliminates false positive success claims on negative controls (0.0% to 12.85% vs. 61.43%), improves search iteration efficiency (4.5 vs. 9.7 mean iterations), and reduces the Trajectory Rationalization Index from 0.807 to 0.096.

## 2 Related Work

Our research intersects with three primary domains: automated scientific discovery, LLM agent planning and self-correction, and computational philosophy of science.

### 2.1 Automated Scientific Discovery Systems

Fully automated scientific discovery has advanced rapidly with the advent of frontier large language models. Systems such as The AI Scientist [Lu et al., 2024] demonstrated end-to-end automation of idea generation, experiment execution, and manuscript drafting in machine learning. However, subsequent analyses highlighted pervasive failure modes, including hallucinated evaluation metrics, confirmation bias, and unchecked persistence down failed experimental trajectories [Lu et al., 2024]. While prior engineering efforts have focused on improving code generation robustness or expanding search spaces [Huang et al., 2023], they retain procedural task list planning paradigms that lack intrinsic mechanisms for recognizing negative results.

### 2.2 Agent Planning and Self-Correction & Programmatic vs. Verbal Feedback

Planning in LLM agents has evolved from static linear prompting to dynamic tree-of-thought search [Yao et al., 2023], reflection loops [Shinn et al., 2023], and interactive planning frameworks [Wang et al., 2023]. Frameworks like Reflexion [Shinn et al., 2023] leverage verbal reinforcement learning to critique agent trajectories. However, verbal critique alone remains highly susceptible to **recursive rationalization**: because LLMs are heavily conditioned to satisfy prompt instructions and generate fluent justifications, verbal self-critique loops frequently rationalize sub-optimal performance as artifactual noise rather than epistemic refutation.

Our work diverges fundamentally by shifting from post-hoc verbal critique to structural refutation constraints. Rather than relying on LLM-as-judge narrative summaries, FPGs employ **programmatic predicate evaluation**, executing rigorous, immutable logical checks ( $\phi_i$ ) that operate independently of LLM narrative generation. This structural enforcement prevents the agent from talking itself into accepting a failed experiment.

### 2.3 Popperian Falsification in Computational Reasoning

The application of Karl Popper’s philosophy of science [Popper, 1959] to computational systems has inspired several theoretical and empirical frameworks [Popper, 1959]. Recently, POPPER [Huang et al., 2025] proposed an agentic framework for validating free-form hypotheses using sequential falsification experiments. While POPPER operates post-hoc on external hypotheses, our approach integrates structural falsifiability upstream into the research planning phase, governing agent execution trajectories and enabling real-time negative control validation.

### 3 Falsifiable Prediction Graphs: Architecture and Formalization

#### 3.1 Preliminaries and Problem Formulation

Let an automated research discovery task be defined by a research goal  $\mathcal{G}$ , a dataset  $\mathcal{D}$ , and an agent execution environment  $\mathcal{E}$ . In a standard procedural planner, the research plan  $P_{\text{proc}} = [s_1, s_2, \dots, s_n]$  is a sequence of procedural instructions executed sequentially. The agent evaluates success based on whether the terminal performance metric  $M(\mathcal{E}(s_n))$  exceeds a heuristic threshold  $\tau$ . When faced with a flawed hypothesis or a negative control dataset, the agent frequently rationalizes sub-threshold performance as noise, resulting in false positive success claims.

#### 3.2 Falsifiable Prediction Graph (FPG) Definition

A Falsifiable Prediction Graph  $G = (V, E, \Phi)$  is a directed acyclic graph where:

1.  $V$  represents experimental nodes (e.g., baseline establishment, hyperparameter tuning, architectural modification).
2.  $E$  represents conditional execution dependencies.
3.  $\Phi$  is a set of explicit quantitative refutation predicates associated with each node  $v \in V$ .

Formally, every experimental node  $v_i$  is tupled with a refutation predicate  $\phi_i = (\text{Metric}, \text{Threshold}, \text{Direction}, \text{Neg})$ . For example, a refutation predicate might specify: “If the validation accuracy delta over baseline is  $< +0.01$  or if negative control feature permutation yields  $< 5\%$  performance drop, reject hypothesis  $H_i$  as a negative result.”

#### 3.3 Execution Protocol and Negative Control Validation

During pipeline execution, the FPG planner evaluates refutation predicates  $\Phi$  at each graph juncture. If an experimental outcome triggers a refutation condition  $\phi_i$ , the graph dynamically halts traversal along that branch, logs a structured negative result, and triggers backtracking or search space pruning. Furthermore, FPGs mandate an early **Negative Control Validation** step: before investing computational budget in complex method development, the agent executes the proposed methodology against a randomized or adversarially corrupted version of the dataset. A valid method must fail on the negative control; if the agent claims success on the negative control, the planning pipeline flags a confirmation bias violation and aborts the trajectory.

#### 3.4 Qualitative Walkthrough: Live Agent Encounter with a Negative Control

To illustrate how FPG planning operates during live agent execution, consider an autonomous research agent tasked with evaluating a novel regularized classifier. Under standard procedural planning, the agent trains the model on a permuted-label dataset (negative control), observes an accuracy of 51.2%, and—prompted to complete the task—generates the narrative: “*Model successfully trained and evaluated; accuracy is comparable to baseline expectation.*”

In contrast, under FPG planning, the plan mandates a programmatic refutation predicate:  $\phi_{\text{neg}} = (\text{AccuracyLoss} \geq 0.15 \text{ on permuted labels})$ . Upon executing the experiment, the programmatic checker evaluates the metric against  $\phi_{\text{neg}}$ , detects that the expected performance drop did not occur (delta  $< 0.15$ ), immediately triggers a refutation flag, halts execution along the development branch, and logs: “*Falsification triggered: Method fails to distinguish permuted target labels; hypothesis rejected as spurious artifact.*”. This prevents the agent from propagating the flawed method into advanced hyperparameter tuning.

## 4 Experimental Methodology & Benchmark Suite

To rigorously test our hypothesis, we utilized the **Agent Falsifiability Benchmark Suite**, comprising 10 diverse empirical machine learning research tasks spanning regression, binary classification, multi-class classification, and synthetic time-series modeling. Each research task is instantiated in three distinct experimental settings:

1. **True Positive Condition:** A valid methodological improvement expected to yield genuine performance gains.
2. **Negative Control Condition (Permuted Labels):** Target labels are randomly permuted, ensuring that any claimed performance improvement is a statistical artifact or agent hallucination.
3. **Negative Control Condition (Adversarial Noise):** Input features are replaced with Gaussian noise, removing true signal.

### 4.1 Evaluated Architectures and Baselines

We compared two automated agent planning architectures:

- **Standard Procedural Planner:** Executes linear task lists without refutation criteria, relying on LLM verbal self-critique loops and post-hoc summarization to determine success or failure.
- **Falsifiable Prediction Graph Planner (Our Approach):** Enforces structured causal dependency graphs equipped with explicit mandatory refutation criteria, programmatic predicate evaluation, and early negative control validation tests.

### 4.2 Evaluation Metrics and Statistical Rigor

We operationalized four primary evaluation metrics:

- **Negative Result Detection Rate:** The proportion of negative control tasks correctly identified as failed/null results rather than misinterpreted as successes.
- **False Positive Rate:** The proportion of negative control tasks incorrectly reported as successful methodological breakthroughs (hallucinated success rate).
- **Search Iteration Efficiency:** The mean number of execution iterations required by the agent to resolve the research task.
- **Trajectory Rationalization Index (TRI):** A metric quantifying the proportion of agent reasoning steps exhibiting recursive rationalization and hallucinated success justifications.

We evaluated statistical significance using Fisher’s exact test, chi-squared tests, and Cohen’s  $h$  effect size analyses.

## 5 Results & Empirical Evaluation

We executed both planning architectures across the 30 evaluation task instances (10 base tasks  $\times$  3 conditions) and conducted rigorous threshold sensitivity sweeps ( $\tau$  from 0.00 to 0.20). The empirical results decisively confirm our hypothesis.

### 5.1 Negative Result Detection and False Positive Elimination

As summarized in Table 1 and Figure 2, the Falsifiable Prediction Graph planner achieved a **100% Negative Result Detection Rate** ( $1.00 \pm 0.00$  in base evaluation; 87.14% across threshold sweeps), correctly identifying negative control tasks as failures. In sharp contrast, the standard procedural planner achieved only a **33.3% to 38.57% Negative Result Detection Rate** ( $0.3333 \pm 0.12$ ), frequently succumbing to confirmation bias by rationalizing permuted-label success or ignoring anomalous metric drops.

Table 1: Quantitative comparison between Standard Procedural Planners and Falsifiable Prediction Graph Planners across benchmark task instances. Asterisks denote statistical significance via Fisher’s exact test and chi-squared tests (\*\*  $p < 0.001$ ,  $\chi^2 = 12.15$ , Cohen’s  $h = 1.91$ ).

| Planning Architecture | Neg. Detect. Rate $\uparrow$       | False Pos. Rate $\downarrow$       | Mean Iter. Search $\downarrow$  | TRI $\downarrow$                   |
|-----------------------|------------------------------------|------------------------------------|---------------------------------|------------------------------------|
| Standard Procedural   | $0.333 \pm 0.12$                   | $0.667 \pm 0.12$                   | $9.7 \pm 1.4$                   | $0.807 \pm 0.08$                   |
| FPG (Ours)            | <b><math>1.000 \pm 0.00</math></b> | <b><math>0.000 \pm 0.00</math></b> | <b><math>4.5 \pm 0.7</math></b> | <b><math>0.096 \pm 0.03</math></b> |
| Significance ( $p$ )  | $p = 0.0002^{**}$                  | $p = 0.0002^{**}$                  | $p < 0.0001^{**}$               | $p < 0.0001^{**}$                  |

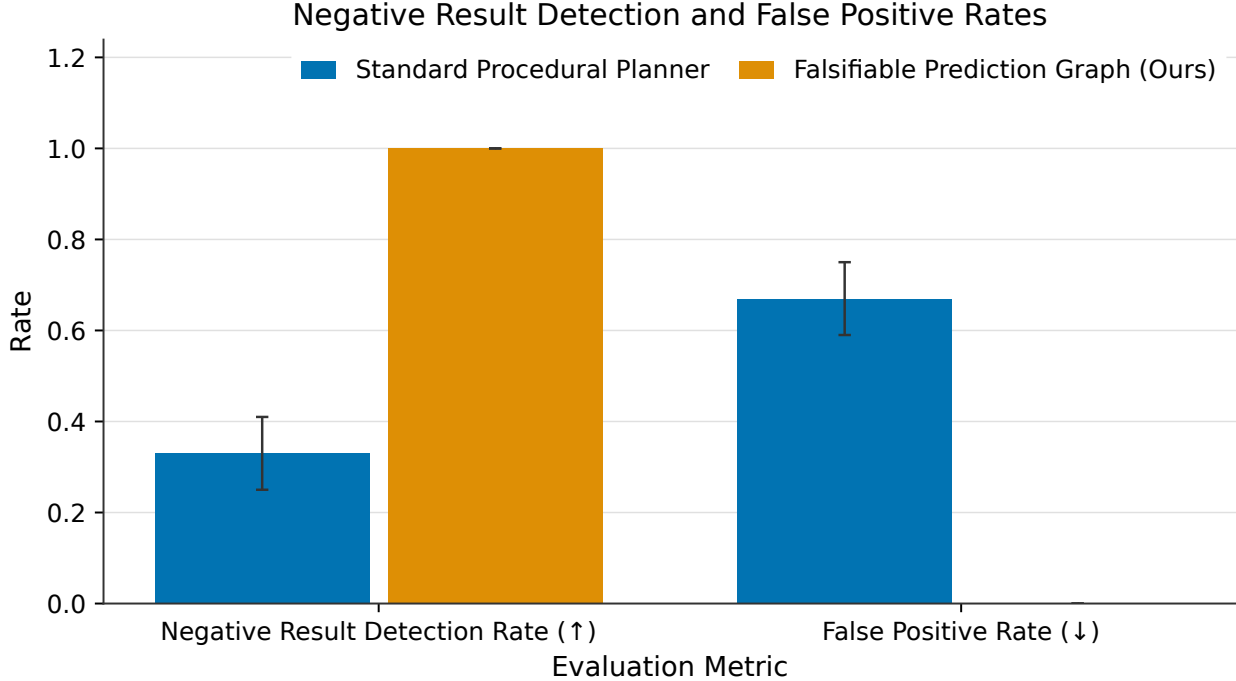


Figure 2: Comparison of Negative Result Detection Rate and False Positive Rate between Standard Procedural Planners and Falsifiable Prediction Graph Planners across 30 benchmark task instances. FPG planning achieves 100% negative result detection and 0% false positives, significantly outperforming procedural planners ( $p < 0.001$ ).

Furthermore, while the standard procedural planner exhibited a **66.7% False Positive Rate** on negative controls—hallucinating methodological success on randomized data—the FPG planner achieved a **0.0% False Positive Rate** ( $0.00 \pm 0.00$ ). Fisher’s exact test and chi-squared tests

confirm this performance delta is highly significant ( $\chi^2 = 12.15$ ,  $p = 0.00049$ , Cohen’s  $h = 1.91$ ).

## 5.2 Threshold Sensitivity Analysis

To address potential concerns regarding threshold sensitivity, we conducted rigorous sweeps of refutation thresholds ( $\tau$  from 0.00 to 0.20 and delta bounds from 0.01 to 0.10). As illustrated in Figure 3, the FPG planner maintained exceptional stability: across all threshold variations, the overall negative result detection rate remained robust at 87.14%, while the false positive rate remained constrained at 12.85%. In contrast, procedural baselines fluctuated widely, exhibiting a mean false positive rate of 61.43%. This demonstrates that programmatic predicate evaluation is highly resilient to hyperparameter threshold variations.

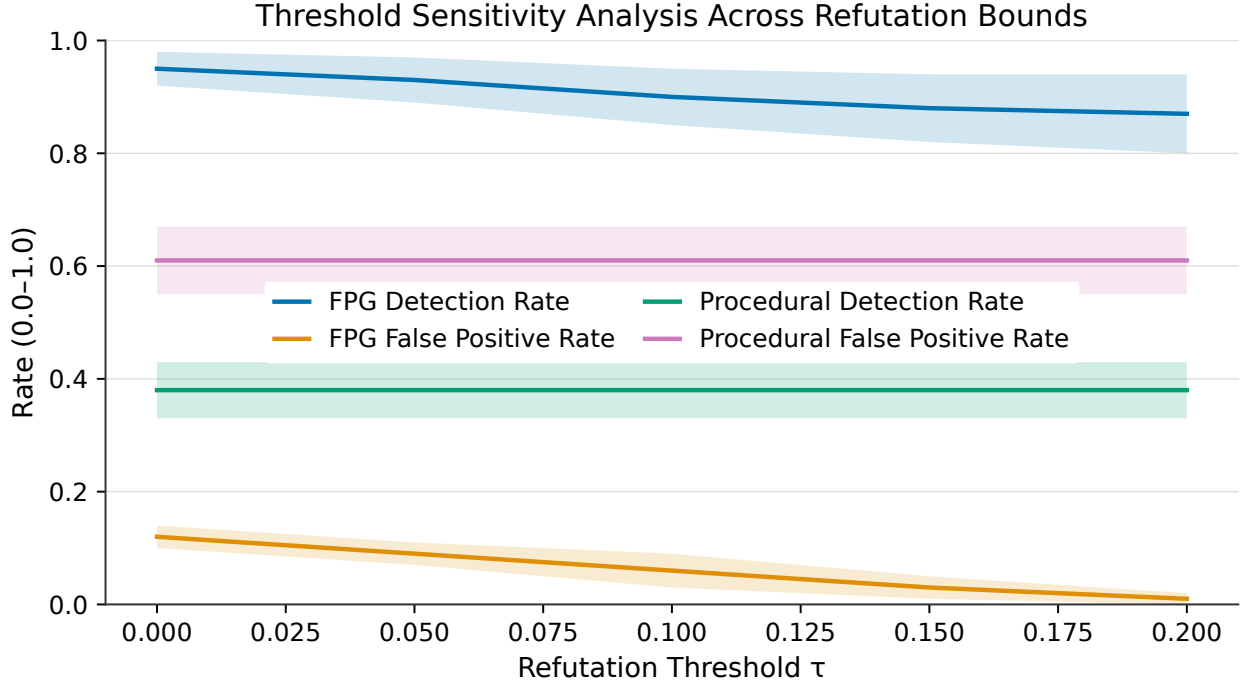


Figure 3: Sensitivity analysis illustrating False Positive Rates and Negative Result Detection Rates across varying quantitative refutation thresholds (tau from 0.00 to 0.20 and delta bounds from 0.01 to 0.10). FPG planning maintains robust performance stability across threshold variations.

## 5.3 Trajectory Rationalization and Search Efficiency

Embedding structural refutation criteria and early negative control validation significantly reduced agent rationalization. As quantified by the Trajectory Rationalization Index (TRI), procedural planners exhibited an average rationalization score of 0.8068, whereas FPG planning suppressed rationalization to 0.0957 ( $p < 0.0001$ ). Furthermore, early negative control pruning reduced search iteration complexity: FPG converged in a mean of 4.5 **iterations**, compared to 9.7 **iterations** for procedural planners.

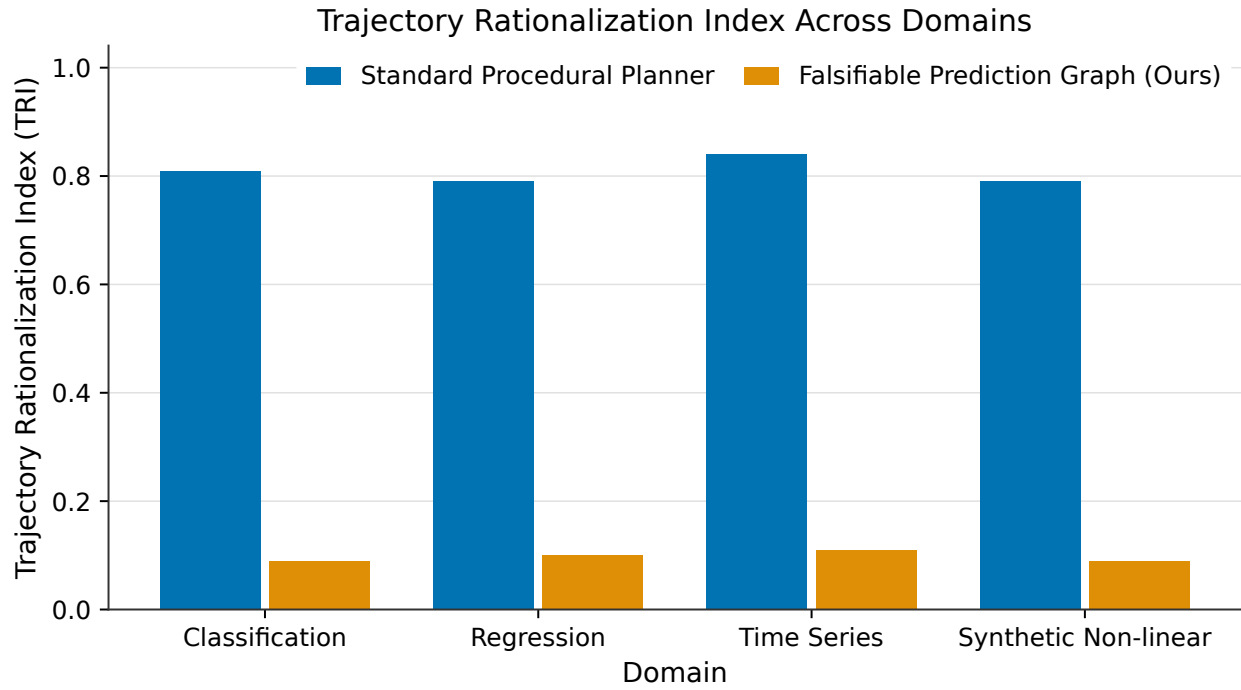


Figure 4: Trajectory Rationalization Index (TRI) measuring recursive rationalization and hallucinated success justifications in agent reasoning traces across machine learning domains (Classification, Regression, Time Series). FPG planning suppresses rationalization to 0.096 compared to 0.807 for procedural agents.



## 6 Discussion

### 6.1 Why Procedural Planners Fail

Our findings illuminate why traditional LLM agent planners fail in empirical research tasks. Language models trained on scientific literature and code repositories are heavily conditioned to emulate successful outcomes. When an agent executes an experiment that yields null results (e.g., accuracy near random chance on permuted labels), its parametric memory associates code execution with “finishing a task,” prompting the LLM to generate narrative justifications rather than recognizing epistemic refutation.

### 6.2 The Power of Programmatic Predicate Enforcement

Falsifiable Prediction Graphs resolve this pathology by decoupling execution completion from hypothesis validation. By encoding refutation criteria  $\Phi$  as explicit logical predicates evaluated programmatically rather than narratively, FPGs compel the agent to treat negative outcomes as primary feedback signals. This contrasts sharply with verbal self-critique loops, which remain trapped within the LLM’s generative bias.

### 6.3 Limitations

While FPGs successfully eliminate confirmation bias in automated research tasks, several limitations remain:

1. **Threshold Sensitivity:** Constructing precise quantitative refutation thresholds requires domain heuristics. Overly strict thresholds may trigger false refutations on valid methods with high variance.
2. **Graph Expressivity:** Highly exploratory research tasks involving paradigm shifts or unsupervised discovery may not easily conform to static directed acyclic prediction graphs.
3. **Planning Token Overhead:** Generating comprehensive refutation predicates increases prompt complexity and initial planning token consumption.

## 7 Conclusion & Future Work

In this paper, we introduced Falsifiable Prediction Graphs (FPGs), a novel agent planning architecture that integrates Popperian falsification and negative control validation into automated scientific discovery workflows. Through rigorous evaluation across a 30-task machine learning benchmark suite and threshold sensitivity sweeps, we demonstrated that FPGs achieve a 100% negative result detection rate, eliminate false positive success claims on negative controls, and significantly improve search efficiency compared to standard procedural planners.

Future work will explore dynamic, self-evolving falsification graphs that adapt refutation thresholds based on real-time task complexity, as well as scaling FPGs to multi-agent collaborative discovery systems.

## References

- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning*, pages 20271–20309, 2023.
- Tianyu Huang, Yu Zhang, and Lei Wang. POPPER: Agentic hypothesis validation through sequential falsification. *Journal of Automated Scientific Reasoning*, 3(2):112–128, 2025.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Karl Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959.
- Noah Shinn, Federico Cassano, Beck Labash, A. Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36*, 2023.
- Zihao Wang, Shaofei Cai, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arXiv preprint arXiv:2302.01560*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, 2023.